

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Part 1:

You and your partner must share each other's Assignment 1 submission.

Part 2:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

```
In [4]: # Your answer here
# 1) We are provided with the binary tree.
# 2) The binary tree has Leaves (the values with no subtrees)
# 3) Our task is to show (print out) the paths connecting the root of the tree with the Leaves.
```

- Create 1 new example that demonstrates you understand the problem. Trace/walkthrough 1 example that your partner made and explain it.

```
In [3]: # Your answer here
# New example:
# Our root is 200.
# - It's Left child is 2 and its right child is 100 (Level-1)
# - The right value of 100 from Level-1 has left child of 2 and right child of 50 (Level-2).
# - The right value of 50 from Level-2 has left child of 2 and right child of 25 (Level-3).
```

```

# - The right value of 25 from Level-3 has left child of 5 and right child of 5.
# Tree demonstration:
#   200
#   |  |
#   2  100
#   |  |
#   2  50
#   |  \
#   2  25
#   |  \
#   5  5
# Expected output: [[200, 2], [200, 100, 2], [200, 100, 50, 2], [200, 100, 50, 25, 5], [200, 100, 50, 25, 5]]

# Partner Example 1:
# 1) Start at the root :document -> (1)
# 2) Visit its children : projects -> (2) and assignments -> (3)
# 3) Then visit the children of projects and assignments in order :4,5 and 6,7 : (children of projects : project1.doc -> (4) and p
# input[1,2,3,4,5,6,7]
# output[[1,2,4],[1,2,5],[1,3,6],[1,3,7]]

# This example could be rephrased in the following way:
# The root value is 1 (Level 0)
# - It's left child is 2 and its right child is 3 (Level-1)
# - The left value of 2 from Level-1 has left child of 4 and right child of 5.
# - The right value of 3 from Level-1 has left child of 6 and right child of 7.
# Tree demonstration:
#   1
#  /  \
# 2    3
# / \  / \
#4  5 6  7
# Expected output: [[1, 2, 4], [1, 2, 5], [1, 3, 6], [1, 3, 7]]

```

- Copy the solution your partner wrote.

```

In [1]: # Your answer here
from collections import deque
from typing import List

# Define the TreeNode class
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

```

```

def bt_path(root: TreeNode) -> List[List[int]]:
    if not root:
        return []

    result = []
    queue = deque([(root, [root.val])]) # Store node and path so far

    while queue:
        node, path = queue.popleft()

        # If it's a leaf node, add the path to result
        if not node.left and not node.right:
            result.append(path)

        # Add left child to queue if it exists
        if node.left:
            queue.append((node.left, path + [node.left.val]))

        # Add right child to queue if it exists
        if node.right:
            queue.append((node.right, path + [node.right.val]))

    return result

# Example Usage
# Construct the binary tree: [1, 2, 2, 3, 5, 6, 7]
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(2)
root.left.left = TreeNode(3)
root.left.right = TreeNode(5)
root.right.left = TreeNode(6)
root.right.right = TreeNode(7)

# Get all paths from root to leaf nodes
output = bt_path(root)
print(output)

```

```
[[1, 2, 3], [1, 2, 5], [1, 2, 6], [1, 2, 7]]
```

- Explain why their solution works in your own words.

```

In [2]: # Your answer here
# The partner's solution is based on the class TreeNode and specifically focuses on the method bt_path.
# This method uses Breadth First Search (BFS) approach to traverse the tree.
# BFS helps to identify all the paths from root to the leaves.

```

```
# More specifically, we are exploring every node once adding it the queue until the leaf node is approached.  
# When the leaf node is approached the specific path from the queue is added to the result.
```

- Explain the problem's time and space complexity in your own words.

```
In [5]: # Your answer here  
# In case we have n nodes in the tree:  
# Time complexity is O(n) as every node is explored 1 time during the BFS  
# Space complexity is O(n) as in the worst case scenario there could be up to n/2 nodes at the last level to store
```

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

```
In [6]: # Your answer here  
# Partner's solution is accurate and to the point. BFS algorithm was implemented properly.  
# I tested the provided examples, my partner's examples, and additional cases, all of which produced correct results. Therefore, t  
# My only recommendation is regarding the examples (test cases). It seems the suggested examples are essentially similar in conten  
# I believe it is crucial to incorporate greater variety and include additional edge cases in the testing process.
```

Part 3:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

```
In [7]: # Your answer here  
# I enjoyed working with the Assignments at this module as they were constructed to replicate the actual software development proc  
# I had to follow the critical steps of  
# (1) defining the task,  
# (2) proposing the test cases,  
# (3) providing the solution,  
# (4) reflecting about the time and space complexity of the crafted solution (in case any inefficiencies are identified, the solut  
# and any alternative solutions,  
# (5) and finally the ability to receive a feedback from other team members (it could be both from Q&A and other peers commenting  
  
# The assignment 1 helped me to train the application of the Merge Sort in the real-life situation (in terms of coding) and practi  
  
# My collaboration with the partner was beneficial too. As I was able to receive valuable feedback from Marjan (partner's name) to  
# there are any improvements/changes to be implemented into my solution and test cases.  
# In return, I was able to explore other problem from the assignment to train and learn in practice BFS and DFS algorithms.
```

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

🔔 **Please review our [Assignment Submission Guide](#)** 🔔 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: `HH:MM AM/PM - DD/MM/YYYY`
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
`https://github.com/<your_github_username>/algorithms_and_data_structures/pull/<pr_id>`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at [#cohort-3-help](#). Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.